https://www.techiedelight.com/bit-hacks-part-1-basic/?fbclid=IwAR39rmIP1O5_GPgHBkOA8oleO3XFOKsU3bYwiZ39gpGksdQKzD1NdS53Arg

Trucos de manipulación de bits

Pregunta	Expresión	Ejemplo
1- ¿ Comprueba si un entero es **par o impar** ?	**& (AND)** **N & 1** devuelve 1 o 0 dependiendo de si n es par o impar.	00010100 & (n = 20) 00000001 (1) ~~~~~~~~ 00000000 00010101 & (n = 21) 00000001 (1) ~~~~~~~~ 00000001
¿ Detectar si dos números enteros tienen **signos opuestos** o no?	**^ (XOR)** **((x ^ y) < 0)** es negativa si x y y tienen signos opuestos **positivo**:bit más a la izquierda **0** **negativo:**bit más a la izquierda **1** **Entero de signos iguales:** XOR establece bit más a la izquierda **0** **Entero de signos opuestos:** XOR establece bit más a la izquierda **1**	00…000100 ^ (x = 4) 00…001000 (y = 8) ~~~~~~~~~~ 00…001100 número positivo 00…000100 ^ (x = 4) 11…111000 (y = -8) ~ ~~~~~~~~ 11…111100 número negativo
Sumar 1 a un número entero	~ (NOT): Invierte el valor de los bit y suma 1 Los números negativos se almacenan en forma de complemento a 2 ~x= -(x + 1) ejemplo: ~2 = -3 -~x= (x + 1) ejemplo: -~2 = 3	Ejemplo: ~2 = -3 Ejemplo: -~2 = 3 00000010 ~ (x = 2) **1**1111101(C_2) (x = -3) **1**1111101(C_2) = 00000011($_{10}$) (x = -3)
Intercambiar dos números sin usar ninguna tercera variable	x ^ x = 0 . **x = x ^ y;** **y = x ^ y;** **x = x ^ y;**	00010100 ^ (x = 20) 00010101 (y = 21) ~~~~~~~~ 00000001 **x = x ^ y;** **(diferencia)** 00000001 ^ (x = 1) 00010101 (y = 21) ~~~~~~~~ 00010100 **y = x ^ y;** 00000001 ^ (x = 1) **00010100** (y = 21) ~~~~~~~~ **00010101** **x = x ^ y;**

https://www.techiedelight.com/bit-hacks-part-2-playing-kth-bit/

Desactiva el k-ésimo bit en un número	<< (desplaza izquierda) a << b Desplaza a la izquierda un número especificado de bits. 1 << (3-1) 00000001 << 2 = **00000100** 00000100 ~ (x = **4**) **11111011**(C_2) n & ~(1 << (k - 1))	n = 20 y k-ésimo = 3 00010100 & (n = 20) 11111011 ~ (1 << (3-1)) ~~~~~~~~ 00010000
Activa el k-ésimo bit en un número	1 << (3-1) 00000001 << 2 = **00000100** n | (1 << (k - 1))	n = 20 y k-ésimo = 3 00010100 | (n = 20) 00001000 (1 << (3-1)) ~~~~~~~~ 00011100
Comprueba si el k-ésimo bit está establecido, es decir activo o 1	n & (1 << (k – 1)) **!= 0**	
Invertir el k-ésimo bit en un número	n ^ (1 << (k – 1))	n = 20 y k-ésimo = 3 00010100 ^ (n = 20) 00001000 (1 << (3-1)) ~~~~~~~~ 00011100 00011100 ^ (n = 20) 00001000 (1 << (3-1)) ~~~~~~~~ 00010100

https://www.techiedelight.com/bit-hacks-part-3-playing-rightmost-set-bit-number/

Desactivación del bit establecido más a la derecha de un número

¿Desactivar el bit establecido más a la derecha de un número?	n & (n-1)	00010100 & (n = 20) 00010011 (n-1 = 19) ~~~~~~~~ 00010000
Comprueba si un entero positivo es una **potencia de 2** sin usar ninguna bifurcación o bucle.	Si al desactivar el único bit, da 0 entonces es potencia de 2, porque Si el número es una potencia de 2, solo tiene un solo bit establecido n & (n-1) =**= 0** El **0** se considera incorrectamente como una potencia de 2 aquí. Para remediar esto, use: **n && !(n & (n - 1));**	00010000 & (n = 16, solo un bit establecido) 00001111 (n-1 = 15) ~~~~~~~~ 00000000
Encuentre la posición del bit establecido más a la derecha	n ^ (n & (n-1)) **int pos = 0;** **while (n != 0)** **{** **n = n >> 1;** **pos++;** **}** **return pos;** **Otra forma sería** **(n & -n) == n** **Return Math.log2(resultado) + 1;**	00010100 & (n = 20) 00010011 (n-1 = 19) ~~~~~~~~ 00010000 00010100 ^ (n = 20) 00010000 ~~~~~~~~ 00000100 el bit establecido más a la derecha nos lo dirá la posición **Otra forma sería** 00…0010100 & (n = 20) 11…1101100(C_2) (-n = -20) ~~~~~~~~~~ 00…0000100
Encuentra la posición del único bit establecido en un número	**Resultado** = n ^ (n & (n-1)) **Return Math.log2(resultado)**	00010000 & (n = 16) 00001111 (n-1 = 15)

	+ 1;	~~~~~~~~ 00000000 log2(16) + 1 = 5

Cálculo de la paridad de un número
Número total de 1 en un número binario
La paridad impar (1) significa un número impar de 1 y la paridad par (0) significa un número par de 1.
	paridad = false;
	while (n != 0) {
	//invertir bandera paridad
	paridad = !paridad;
	// apagando bit establecido derecha
	n = n & (n - 1);
	}
	
	return paridad	

Cual es la diferencia de un número con su antecesor?

n-1:

- tendrá todos los **bits invertidos después del bit establecido más a la derecha de n** (incluido el bit establecido más a la derecha).
- Tendrá todos los **bits iguales antes del bit establecido** más a la derecha de n

ejemplo:

11111111	255
11111110	254
11111101	253

Cual es la diferencia de un número con su predecesor?

n+1:

- tendrá **todos los bits iguales antes del bit "NO establecido"** más a la derecha de n
- tendrá **todos los bits invertidos después del bit "NO establecido"** más a la derecha de n (incluido el bit "NO establecido" más a la derecha).

ejemplo:

11111111	255
11111110	254

11111101 253

Invertir minusculas y mayúscula	letra ^ (1 << 5)	1100001 ^ (a) 0100000 (6 bit) ---------- 1000001 (A) 1000001 ^ (A) 0100000 (6 bit) ---------- 1100001 (a)
Convierte caracteres en mayúsculas a **minúsculas**.	letra | (1 << 5)	1100001 | (a) 0100000 (6 bit) ---------- **1100001 (a)** 1000001 | (A) 0100000 (6 bit) ---------- **1100001 (a)**
Convierte caracteres en minúsculas a **mayúsculas**.	letra **&** ~(1 << 5)	0100000 (1 << 5) 1011111 ~(1 << 5) 1100001 **&** (a) 1011111 ~(1 << 5) ---------- **1000001 (A)** 1000001 **&** (A) 1011111 ~(1 << 5) ---------- **1000001 (A)**
Encuentra la posición de una letra en el alfabeto	letra **&** 31	**1000001 & (A)** 0011111 (31) ~~~~~~~~ 0000001 (1) 1100100 **&** (d) 0011111 (31) ~~~~~~~~ 0000100 (4)

Si cogemos todos los caracteres en mayúsculas (A-Z) y todos los minúsculas (a-z) e imprimimos su código en binario, observamos que **todos los bits son iguales**, **excepto el sexto bit** (contando desde la derecha):

a = 1100001 b = 1100010 c = 1100011 d = 1100100 e = 1100101 f = 1100110 g = 1100111 h = 1101000 i = 1101001 j = 1101010 k = 1101011 l = 1101100 m = 1101101 n = 1101110 o = 1101111 p = 1110000 q = 1110001 r = 1110010 s = 1110011 t = 1110100 u = 1110101 v = 1110110 w = 1110111 x = 1111000 y = 1111001 z = 1111010

A = 1**0**00001 B = 1000010 C = 1000011 D = 1000100 E = 1000101 F = 1000110 G = 1000111 H = 1001000 I = 1001001 J = 1001010 K = 1001011 L = 1001100 M = 1001101 N = 1001110 O = 1001111 P = 1010000 Q = 1010001 R = 1010010 S = 1010011 T = 1010100 U = 1010101 V = 1010110 W = 1010111 X = 1011000 Y = 1011001 Z = 1011010

Minúsculas: tienen el sexto bit a **1** a = 1**1**00001
Mayúsculas: tienen el sexto bit a **0** . A = 1**0**00001

Haciendo un XOR de un carácter con el carácter espacio cuyo código es el **32 en decimal(128; 64; 32; 16; 8; 4; 2; 1),** por lo tanto 100000 en binario podemos hacer la conversión, ya que esta operación invertirá el bit 6.

Calcular el signo de un entero		
Calcule el valor absoluto entero (abs) sin bifurcarse		
Calcule el mínimo (min) o el máximo (max) de dos números enteros sin bifurcarse		
Versiones rápidas y sucias:		
Determinar si un número entero es una potencia de 2		
Signo que se extiende desde un ancho de bits constante		
Signo que se extiende desde un ancho de bits variable		
Signo que se extiende desde un ancho de bits variable en 3 operaciones		
Establecer condicionalmente o borrar bits sin ramificación Negar condicionalmente un valor sin bifurcar		
Combinar bits de dos valores según una máscara		
Conjunto de bits de conteo (forma ingenua)		
Bits de conteo establecidos por la tabla de búsqueda		
Juego de conteo de bits, a la manera de Brian Kernighan		
Bits de conteo establecidos en palabras de 14, 24 o 32 bits		

usando instrucciones de 64 bits		
Conjunto de bits de conteo, en paralelo		
Cuente los bits establecidos (rango) desde el bit más significativo hasta una posición dada		
Seleccione la posición del bit (desde el bit más significativo) con el conteo (rango) dado		
Cálculo de la paridad de forma ingenua		
Calcular paridad por tabla de búsqueda		
Calcule la paridad de un byte usando multiplicación de 64 bits y división de módulo		
Calcular la paridad de palabra con una multiplicación		
Calcular la paridad en paralelo		
Intercambio de valores con resta y suma		
Intercambio de valores con XOR		
Intercambio de bits individuales con XOR		
Bits inversos de la manera obvia		
Bits inversos en palabra por tabla de búsqueda		
Invierta los bits en un byte con 3 operaciones (multiplicación de 64 bits y división de módulo):		
Invierta los bits en un byte con 4 operaciones (multiplicación de 64 bits, sin división):		
Invierta los bits en un byte con 7 operaciones (sin 64 bits):		
Invierta una cantidad de N bits en paralelo en operaciones de 5 * lg(N):		
Calcule la división del módulo por 1 << s sin un operador de		

división		
Calcule la división del módulo por (1 << s) - 1 sin un operador de división		
Calcule la división del módulo por (1 << s) - 1 en paralelo sin un operador de división		
Encuentre la base logarítmica 2 de un número entero con el MSB N establecido en operaciones O (N) (la forma obvia)		
Encuentre el registro de enteros base 2 de un entero con un flotante IEEE de 64 bits		
Encuentre la base logarítmica 2 de un número entero con una tabla de búsqueda		
Encuentre el logaritmo en base 2 de un entero de N bits en operaciones O(lg(N))		
Encuentre la base logarítmica 2 de un entero de N bits en operaciones O(lg(N)) con multiplicación y búsqueda		
Encuentra la base de registro de números enteros 10 de un número entero		
Encuentre la base de registro de enteros 10 de un entero de la manera obvia		
Encuentre la base de registro de enteros 2 de un flotador IEEE de 32 bits		
Encuentre la base de registro de enteros 2 de la raíz pow (2, r) de un flotante IEEE de 32 bits (para el entero r sin signo)		
Cuente los bits cero consecutivos (traseros) a la derecha linealmente		
Cuente los bits cero consecutivos (traseros) a la derecha en paralelo		

Cuente los bits cero consecutivos (traseros) a la derecha mediante búsqueda binaria		
Cuente los bits cero consecutivos (traseros) a la derecha convirtiéndolos en un flotador		
Cuente los bits cero consecutivos (traseros) a la derecha con división de módulo y búsqueda		
Cuente los bits cero consecutivos (traseros) a la derecha con multiplicar y buscar		
Redondee a la siguiente potencia más alta de 2 mediante lanzamiento flotante		
Redondea a la siguiente potencia más alta de 2		
Intercalar bits de forma obvia		
Intercalar bits por búsqueda de tabla		
Intercalar bits con multiplicación de 64 bits		
Intercalar bits por números mágicos binarios		
Determinar si una palabra tiene un byte cero		
Determinar si una palabra tiene un byte igual a n		
Determinar si una palabra tiene un byte menor que n		
Determinar si una palabra tiene un byte mayor que n		
Determinar si una palabra tiene un byte entre m y n		
Calcule la siguiente permutación de bits lexicográficamente		

Problemas sencillos sobre manipulaciones de bits y algoritmos bit a bit

Dado un número, encuentre el mayor número menor que el número dado que sea potencia de dos o encuentre el número de bits más significativo. Ejemplo: *El mayor número que es una potencia de 2 menor que 10 es 8*	`Msb = parseInt(Math.log2(n))` `return (1 << msb);`	
Posición del bit establecido más a la derecha	bit = parseInt(Math.log2(n & -n)+1)	
Obtener la posición del bit desactivado más a la derecha	bit=parseInt(Math.log2(~n & -~n)+1)	
Compruebe si el K-th bit está establecido o no	(n & (1<<(k)))!=0	
Compruebe si el bit en la posición dada está activado o desactivado	(n & (1<<(k-1)))!=0	
Encuentra la posición del único bit establecido		
Comprobar el desbordamiento de enteros		
Encuentre XOR de dos números sin usar el operador XOR		
Comprobar si dos números son iguales sin usar operadores aritméticos y de comparación		

Detectar si dos números enteros tienen signos opuestos		
¿Cómo intercambiar dos números sin usar una variable temporal?		
Campesino Ruso (Multiplica dos números usando operadores bit a bit)		

Problemas mediano sobre manipulaciones de bits y algoritmos bit a bit

Intercambiar bits en un número dado		
El menor de tres enteros sin operadores de comparación		
Calcule el mínimo o el máximo de dos números enteros sin bifurcarse		
La potencia más pequeña de 2 mayor o igual que n		
Programa para encontrar la paridad		
Comprobar si la representación binaria de un número es palíndromo		
Generar códigos grises de n bits		
Comprobar si un número dado es escaso o no		
Algoritmo de Euclides cuando las operaciones % y / son costosas		
Calcula el cuadrado de un número sin usar *, / y pow()		
Copiar bits establecidos en un rango		
Comprobar si un número es sombrío		
Conversión de gris a binario y de binario a gris		

Problemas dificiles sobre manipulaciones de bits y algoritmos bit a bit

Siguiente número más alto con el mismo número de bits establecidos		
Encuentre el subarreglo máximo XOR en un arreglo dado		
Encuentre la secuencia más larga de 1 en representación binaria con un giro		
Números más pequeños y más grandes más cercanos (o siguientes) con el mismo número de bits establecidos		
Enmascaramiento de bits y programación dinámica | Problema del vendedor ambulante		
Calcule la paridad de un número usando XOR y búsqueda de tabla		
Cifrado XOR cambiando el texto sin formato		
Cuente los pares en una matriz que tienen al menos un dígito común		
Programa de Python para convertir flotante a binario		
Algoritmo de multiplicación de Booth		
Número de pares con Concatenación Pandigital		
Encuentra el n-ésimo número cuya representación binaria es un palíndromo		
Encuentre los dos elementos que no se repiten en una matriz de elementos que se repiten/Números únicos 2		

https://www.geeksforgeeks.org/find-two-non-repeating-elements-in-an-array-of-repeating-elements/?ref=lbp

Algoritmos bit a bit

https://www.geeksforgeeks.org/bitwise-algorithms/?ref=lbp

Estructura de datos tipo abstractos

https://www.geeksforgeeks.org/learn-data-structures-and-algorithms-dsa-tutorial/?utm_campaign=DSA%20Page%20Tracker&utm_content=Course%20Page&utm_id=DSA-Page-Tracker&utm_medium=Landing%20Page%20Click&utm_source=Website&utm_term=DSA%20Page%20Promo